Agent Postmortem Generator

Part 4: Analysis Engine

One-line purpose: Deterministically reconstruct the failure chain from a canonical trace and identify the first meaningful fault with evidence the rest of the system can trust.

---

1. Executive Overview

The Analysis Engine is the reasoning core of **Agent Postmortem Generator**. It consumes a validated, normalized **Canonical Trace Model** and converts it into a structured **Analysis Result** that explains:

- what failed,
- where the failure propagated,
- which event is the first meaningful fault,
- and what evidence supports that conclusion.

This module exists because raw execution traces are too noisy for direct postmortem generation. Agent runs contain retries, nested tool calls, partial output, cancellations, exceptions, and unrelated diagnostic noise. Humans can manually trace the failure path, but that does not scale and it is error-prone. The Analysis Engine makes that reasoning deterministic, repeatable, and testable.

This module sits directly after the **Canonical Trace Model** and directly before the **Postmortem Generator**. It must not format prose, render diagrams, or speculate beyond the evidence available in the canonical trace.

---

2. Problem Statement

The module solves a very specific problem: **turn a normalized trace into a defensible explanation of failure**.

A trace is not yet an explanation. It is an event log with structure. The hard part is deciding:

- which event actually ended the run,
- which earlier event introduced the defect,
- whether a later failure is only a symptom,
- and how to describe causality without inventing facts.

This is hard because execution traces often have:

- concurrent branches,

- retries that hide the original fault,
- missing or delayed timestamps,
- repeated error messages,
- cancellations caused by upstream failure,
- and partial traces where the final state is ambiguous.

The MVP needs this module because the project's value depends on one thing: **turning messy trace data into a stable reasoning artifact**. Without this module, the rest of the system would either produce shallow summaries or lean on heuristics that are impossible to trust.

---

3. Goals

What success looks like

Goal	Description
Deterministic analysis	The same canonical trace always produces the same analysis result.
Evidence-based conclusions	Every major conclusion can be traced back to one or more specific trace events.
Failure-chain reconstruction	The module can identify the terminal failure and the upstream chain that led to it.
First meaningful fault identification	The module can isolate the earliest event that materially introduced the failure.
Clear ambiguity handling	When evidence is incomplete, the module reports uncertainty instead of guessing.
Stable interface	Downstream modules receive a predictable structured result, not raw analysis chatter.

What this module must achieve

- Consume only the canonical trace model.
- Produce a structured analysis result suitable for the postmortem generator.
- Preserve provenance for every important inference.
- Handle incomplete or awkward traces without collapsing into nondeterminism.
- Remain small enough for a 2 to 3 engineer MVP team to maintain.

---

4. Non-Goals

This module explicitly does **not**:

- generate the final postmortem prose,
- render Markdown, Mermaid, HTML, or PDF,
- talk to remote services,
- call LLMs to "help" interpret the trace,
- import framework-specific agent objects,
- normalize raw input,
- parse JSON files,
- or own the adapter boundary.

Later versions may extend analysis with richer visualization, multi-run comparison, or optional advisory signals. Those belong after the MVP and must not complicate the first implementation.

---

5. Responsibilities

What this module owns

Owned responsibility	Why it belongs here
Build a deterministic execution view	Analysis needs a stable structure over the canonical events.
Identify terminal failure	The analysis must know which event ended the run.
Reconstruct causal sequence	The module traces the failure backward through causal relationships.
Classify failure patterns	Retries, cancellations, timeouts, tool failures, and missing outputs need consistent labels.
Select the first meaningful fault	This is the core judgment the postmortem depends on.
Attach evidence and diagnostics	Downstream must see why each conclusion was reached.
Surface ambiguity	Missing or conflicting data must be reported, not hidden.

What this module must never own

Must never own	Why it is forbidden
Raw parsing	That belongs in the Parser and Validation Engine.
Canonical normalization	That belongs in the Normalization Engine.
Narrative writing	That belongs in the Postmortem Generator.
Presentation concerns	Format, layout, and rendering are output-layer concerns.
Framework-specific semantics	Those must be removed before analysis begins.
Probabilistic interpretation as truth	LLM output is not a source of truth.
External enrichment	No network lookups, remote graph queries, or service calls.

---

6. Inputs

The Analysis Engine accepts a **Canonical Trace Model** instance plus optional analysis configuration.

Expected inputs

Input	Description
Canonical trace	The normalized project-owned trace structure.
Optional analysis policy	Small deterministic knobs such as ordering preferences or classification thresholds, if the MVP exposes them.
Optional run metadata	Local execution context such as source file name or import context, used only for diagnostics.

Validation rules

The Analysis Engine assumes the canonical trace has already passed validation, but it still enforces a small set of contract checks:

- Required identifiers must exist for all referenced nodes.
- Parent, child, and causal references must resolve within the trace.
- Event ordering fields must be present or derivable by deterministic fallback.

- The trace must not contain cycles in any graph that is supposed to be acyclic.
- Required status fields must be set for terminal events.

If these contract checks fail, the module should raise an **internal contract violation** rather than attempting clever recovery. That is a bug in the upstream pipeline, not a signal to improvise.

Assumptions

- The canonical trace is framework agnostic.
- The trace has stable event identifiers.
- The normalization layer has removed most schema noise.
- Missing data will still happen, but it should be semantically interpretable.

Malformed input handling

Malformed or incomplete canonical traces should be divided into two classes:

1. **Recoverable incompleteness**: missing optional fields, ambiguous ordering, partial causal links.
2. **Structural contract violations**: broken references, invalid graph shape, impossible statuses.

Recoverable incompleteness should produce an analysis result with warnings and reduced certainty. Structural contract violations should fail fast.

---

7. Outputs

The module outputs a structured **Analysis Result**. It does not output prose.

Exact outputs

Output artifact	Description
Analysis result	The primary structured object consumed by the Postmortem Generator.
Failure chain	Ordered chain of events or causal nodes leading to the terminal failure.
Fault candidate(s)	One or more candidate root or first-fault events with deterministic ranking.

Output artifact	Description
Evidence set	Event references, fields, and relationships used to justify each conclusion.
Analysis diagnostics	Warnings, ambiguity notes, unsupported patterns, and contract issues.
Derived execution view	A stable internal representation of the trace for analysis steps.

Human-readable artifacts

This module should **not** produce user-facing prose in the MVP. Any readable explanation should be limited to terse diagnostics suitable for logs or debug output. The real explanation belongs downstream.

---

8. Internal Design

The Analysis Engine should be structured as a small deterministic pipeline rather than a monolithic "analyzer."

Main subcomponents

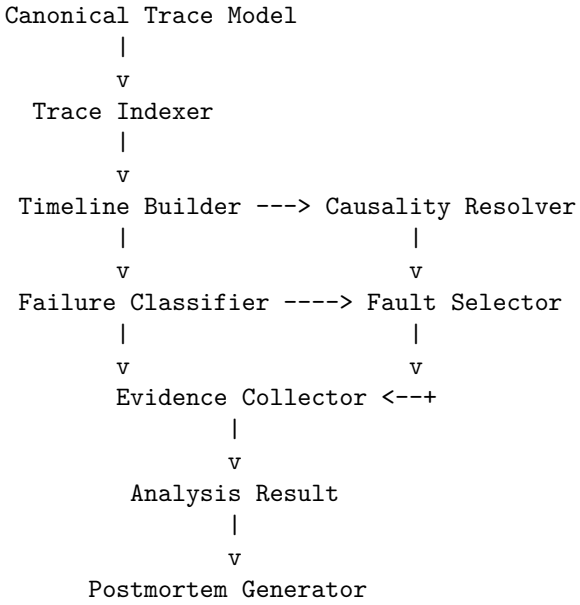
Subcomponent	Responsibility
Trace Indexer	Builds efficient lookups over nodes, edges, statuses, and timestamps.
Timeline Builder	Produces a stable ordering of events and branches.
Causality Resolver	Derives causal links from explicit relations and trace structure.
Failure Classifier	Labels terminal failures and intermediate fault types.
Fault Selector	Chooses the first meaningful fault using deterministic precedence rules.
Evidence Collector	Gathers the exact trace facts that support the analysis.
Diagnostics Assembler	Records warnings, ambiguity, and unsupported situations.

Workflow

1. Read the canonical trace.

2. Build indexes for fast deterministic lookup.
3. Construct a stable execution timeline.
4. Resolve explicit and implicit causal relationships.
5. Detect terminal failure or non-success termination.
6. Walk backward through the causal chain.
7. Classify candidate faults.
8. Select the first meaningful fault.
9. Attach evidence and diagnostics.
10. Emit the final analysis result.

Data flow



Important internal decisions

- Keep the pipeline explicit. Hidden inference chains are hard to test.
- Prefer event identity and explicit links over guessed semantic meaning.
- Use deterministic tie-breakers everywhere.
- Preserve ambiguity rather than flattening it into a fake answer.
- Keep the rule set small enough to explain in the documentation.

---

9. Data Model

The Analysis Engine should operate on a derived model, not on raw canonical objects directly. The derived model is a more analysis-friendly view of the same

information.

Core entities

Entity	Purpose	Key fields
Analysis Result	Top-level output of the module	run_id, terminal_failure, first_fault, failure_chain, evidence, diagnostics
Execution Node View	Indexed view of a canonical event	node_id, event_type, status, timestamps, parent_ids, child_ids
Causal Edge	Explicit or derived relationship between nodes	from_node_id, to_node_id, edge_kind, source_of_truth
Failure Chain	Ordered path used in analysis	ordered_node_ids, terminal_node_id, propagation_steps
Fault Candidate	Candidate first-fault event	node_id, reason, precedence_rank, supporting_evidence
Evidence Item	Fact supporting a conclusion	field_path, value, source_node_id, note
Diagnostic Issue	Warning or analysis limitation	severity, code, message, related_node_ids

Relationships

- An **Analysis Result** contains one **Failure Chain** and one selected **first fault**, plus zero or more alternative candidates.
- A **Failure Chain** consists of ordered **Execution Node Views** connected by **Causal Edges**.
- A **Fault Candidate** must reference at least one **Evidence Item**.
- A **Diagnostic Issue** may refer to any node, edge, or candidate affected by ambiguity.

Why this model exists

The derived model prevents the module from re-deriving the same lookups repeatedly and keeps the core logic understandable. It also makes fixtures easier to build because the analysis can be tested against stable, explicit views of the canonical trace.

10. Algorithms / Processing Logic

The module should follow a deterministic processing sequence.

Step-by-step processing

Step 1: Build stable ordering

Establish a total order over relevant events using the following precedence:

1. explicit sequence number if available,
2. normalized start timestamp,
3. normalized end timestamp,
4. canonical event identifier as final tie-breaker.

If two events are still indistinguishable after these rules, retain both in a stable sorted group and flag the ambiguity.

Step 2: Identify terminal outcome

Determine the final outcome of the run:

- successful completion,
- explicit failure,
- cancellation,
- timeout,
- partial termination,
- or structurally incomplete end state.

The Analysis Engine is primarily used for failed traces, but it must still classify non-failure terminations deterministically so the pipeline does not break when input quality varies.

Step 3: Build the causal frame

Use explicit parent-child edges, failure references, retry links, cancellation causes, and dependency annotations from the canonical trace to build a causal subgraph.

Do **not** infer causality from vague semantic similarity. If the canonical model does not provide enough evidence, the analysis must say so.

Step 4: Locate the terminal failure node

Find the event that best explains the terminating condition. A terminal failure is usually one of:

- a task or tool call that returned an error,
- an explicit exception event,
- a timeout event,

- a cancellation caused by upstream failure,
- or an unrecoverable missing dependency.

Step 5: Walk backward to the first meaningful fault

Traverse upstream through causal relationships to locate the earliest event that materially introduced the failure.

The first meaningful fault is **not** the last symptom. It is the earliest event in the chain that a reasonable engineer would fix first because it introduced the problem, even if later events made the failure visible.

Step 6: Apply deterministic precedence rules

When multiple candidates exist, prefer the one with the highest precedence under a fixed rule order:

Precedence	Candidate type	Reason
1	Explicit upstream exception with direct propagation	Strongest evidence of origin.
2	Failed tool call or task step with concrete error	Usually the first actionable defect.
3	Timeout at the earliest failing boundary	Often the initiating cause when supported by chain evidence.
4	Cancellation caused by prior fault	Usually a symptom, not the origin.
5	Missing output or invalid state discovered later	Often downstream manifestation.

If two candidates have equal precedence, prefer the earlier event in the stable timeline. If they still tie, prefer the event with the most direct evidence references.

Step 7: Collect evidence

For every selected conclusion, gather the minimum necessary evidence:

- event identifiers,
- relevant status fields,
- causal references,
- timestamps,
- retry counts,
- error text if present,
- and any relation that directly supports the chain.

Step 8: Emit diagnostics

If the trace is incomplete, ambiguous, or contains unsupported patterns, preserve that information explicitly in diagnostics.

Edge cases

Parallel failure branches

If two branches fail independently, the module must not collapse them into one synthetic cause. It should either:

- select the branch that actually terminated the run,
- or return multiple fault candidates with a clear dominance rule.

Retry loops

Retries are usually symptoms of the same underlying fault. The analysis should treat repeated failed attempts as part of one causal path unless there is clear evidence of a distinct underlying cause.

Missing timestamps

Missing timestamps are not fatal if event ordering can be derived from sequence numbers or stable identifiers. If no reliable order exists, analysis must mark the chain as partially ordered.

Cycles or invalid causal references

The canonical model should prevent these, but analysis must detect them and fail with a structural contract violation.

Soft failures and cancellations

A cancellation is not automatically the fault. The module must distinguish between the cancelling event and the root event that forced cancellation.

Deterministic behavior requirements

- No randomness.
- No model sampling.
- No heuristic drift across runs.
- No dependence on dictionary iteration order or filesystem order.
- Every ambiguity must resolve via explicit precedence or remain flagged as unresolved.

---

11. Tradeoffs

Simplicity vs flexibility

The MVP should favor simplicity. A compact rule set is easier to understand, test, and review. A highly configurable analysis DSL would be overkill and would create a maintenance burden before the project has enough real-world traces to justify it.

Generality vs specificity

The module should be general enough to work across agent frameworks after normalization, but specific enough to recognize common failure shapes: exception propagation, failed tool calls, retries, timeouts, and cancellations.

Determinism vs heuristic inference

Determinism wins. Heuristic inference may be tempting, especially when traces are incomplete, but the project's credibility depends on predictable behavior. If the evidence is weak, the module should say so instead of guessing.

Why this design is preferred

This design keeps the module explainable, testable, and bounded. It gives the downstream report generator a stable analytical foundation without sneaking in a probabilistic black box.

---

12. Risks and Failure Modes

Hidden complexity

Failure analysis looks simple until concurrent branches and retries show up. The hidden complexity is not in parsing data. It is in deciding what counts as origin versus symptom.

Ambiguous cases

- multiple near-simultaneous failures,
- repeated retries with the same error text,
- cancellation after partial progress,
- partial traces that end without an explicit terminal fault,
- and traces where structural ordering exists but causal ordering is weak.

Unsupported inputs

The module should reject or flag traces that lack the minimum viable canonical structure:

- unresolved node references,
- broken causal edges,
- incoherent statuses,
- or impossible graphs.

Maintenance risks

- Analysis rules drifting into framework-specific special cases.
- Too many ad hoc exceptions for one-off traces.
- Overfitting to a narrow fixture set.
- Allowing diagnostics to become a second hidden output contract.

The main defense is a disciplined rule set, a narrow surface area, and golden fixtures for regressions.

---

13. Testing Strategy

The Analysis Engine must be heavily tested because it is the reasoning core.

Unit tests

Cover the smallest deterministic behaviors:

- event ordering and tie-breaking,
- terminal failure classification,
- causal edge resolution,
- precedence rule application,
- evidence collection,
- ambiguity flagging,
- and diagnostic generation.

Integration tests

Test the module against realistic canonical traces that represent end-to-end trace shapes:

- single failure path,
- retry-heavy failure,
- cancellation after upstream failure,
- timeout after partial progress,
- multiple branch failures,

- and incomplete traces.

Fixture-driven tests

Use golden fixtures with expected Analysis Result outputs. Fixtures should be small but explicit. The output should be asserted structurally, not only by text snapshot, so regressions in causal selection are visible.

Regression protection

Every bug discovered in analysis should become a permanent fixture. If a branch ordering bug or root-cause misclassification occurs once, the exact case should be preserved as a regression test.

---

14. Implementation Checklist

1. Define the Analysis Result contract and keep it narrow.
2. Define the derived execution view used only inside this module.
3. Implement stable event ordering with explicit tie-breakers.
4. Implement causal graph extraction from canonical relations.
5. Implement terminal failure classification.
6. Implement backward traversal to candidate faults.
7. Encode deterministic precedence rules for candidate selection.
8. Implement evidence collection for each conclusion.
9. Implement diagnostics for ambiguity and partial data.
10. Build fixture-driven unit tests for every rule branch.
11. Build integration tests for representative trace shapes.
12. Add regression fixtures for every real bug.
13. Verify that no framework-specific concepts leak into the output contract.
14. Document the supported and unsupported cases clearly.

---

15. Future Expansion

The natural expansions after MVP are the ones that preserve deterministic structure:

- richer causal graph rendering data,
- multi-run comparison analysis,
- optional trace annotations from adapters,
- broader termination classification,
- and better diagnostics for incomplete traces.

Later versions may also expose visualization-friendly exports for execution graphs. That belongs downstream of this module and should not change the core selection logic.

A rule-based configuration layer may eventually make sense if the project accumulates enough diverse trace shapes. For MVP, that is unnecessary overhead.

---

16. Final Recommendation

This design is **MVP-ready** if the implementation stays disciplined about scope.

The simplest version is the right version here:

- a small deterministic analysis pipeline,
- a narrow structured result contract,
- explicit evidence capture,
- and strict handling of ambiguity.

The one simplification still worth preserving is this: **do not add a generic rule engine yet**. Encode the precedence rules directly and keep them readable. The project does not need a mini inference platform before it has a stable trace corpus. That would be architectural vanity in a trench coat.

Appendix: Internal Dependency Shape

```
Canonical Trace Model
        |
        v
  Analysis Engine
        |
        v
 Analysis Result
        |
        v
 Postmortem Generator
```

The module depends on the canonical model as its only source of truth and must remain fully framework-agnostic. Everything else flows from that constraint.